



EcoBag Addis

E-Commerce Database

A complete B2C e-commerce database for an eco-friendly paper bag platform in Addis Ababa — from schema design to stored procedures.

GROUP MEMBERS

#	Full Name	Student ID
1	Kibur Fantahun	ugr-7148-16
2	Ephraim Tefera	ugr-9472-16
3	Yordanos Ketema	ugr-9866-16
4	Melat Tamiru	ugr-4608-16
5	Hilina Melese	ugr-2138-16
6	Getachew Werku	ugr-4034-16

Instructor: Miss Lili | **Course:** Database Systems II | **Academic Year:** 2026

1. System Overview

This section gives a high-level picture of what EcoBag Addis is, why it was built, and how its five core database tables relate to one another.

EcoBag Addis is an online platform that sells environmentally friendly paper bags as sustainable alternatives to plastic packaging. Customers can browse products, place orders, and complete payments entirely through the system. The database captures every stage of this process — from customer registration through to payment confirmation.

Entity Relationship Overview

The database is structured around five core tables. Each table represents a distinct real-world concept, and they are linked together through foreign key relationships that enforce data integrity across the system.

- **Customers** — stores registered buyer profiles.
- **Products** — the catalog of eco-friendly bags available for sale.
- **Orders** — a customer's purchase request, referencing back to Customers.
- **OrderItems** — the individual line items within an order, referencing both Orders and Products.
- **Payments** — the payment transaction tied to a specific order.

In short: a *Customer* places an *Order*, which has one or more *OrderItems* (each linked to a *Product*), and is settled through a single *Payment*.

System Scope

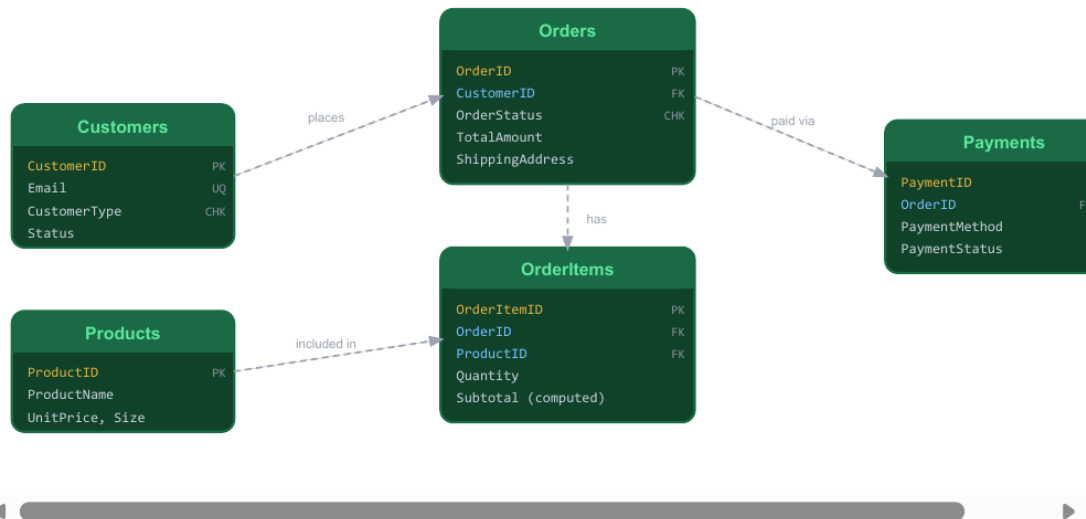
The database supports the following business functions:

- Customer registration and profile management
- Product catalog with sizes, colors, and materials
- Full order lifecycle: Pending → Processing → Shipped → Delivered
- Payment tracking with transaction references
- Retail, Wholesale, and Individual customer types



Entity Relationship Overview

How the 5 core tables relate to each other



Normalization (3NF)

The schema is normalized to Third Normal Form (3NF) to eliminate redundancy and ensure data consistency. Specifically:

- **1NF**: All columns hold atomic values with no repeating groups.
- **2NF**: Every non-key column is fully dependent on the primary key — no partial dependencies exist.
- **3NF**: No transitive dependencies — non-key columns depend only on the primary key, not on other non-key columns.

All primary keys use VARCHAR strings rather than auto-increment integers, giving business-meaningful identifiers such as CUST001 or ORD202505001. Referential integrity is enforced throughout via foreign key constraints.

2. Table Schemas

This section describes the structure of each table — the columns it contains, their data types, and the constraints that govern what data is allowed.

Customers Table — 10 columns

The Customers table is the starting point of the system. Every order must be linked to an existing customer, so this table is the root of the relational hierarchy. Key constraints include a unique email address and a status check that limits accounts to Active, Inactive, or Suspended.

Column	Type	Constraint
CustomerID	VARCHAR(10)	PRIMARY KEY
FirstName	VARCHAR(50)	NOT NULL
LastName	VARCHAR(50)	NOT NULL
Email	VARCHAR(100)	UNIQUE, NOT NULL
PhoneNumber	VARCHAR(15)	NOT NULL
Address	VARCHAR(255)	NOT NULL
City	VARCHAR(50)	NOT NULL
RegistrationDate	DATE	DEFAULT GETDATE()
CustomerType	VARCHAR(20)	CHECK IN ('Retail','Wholesale','Individual')
Status	VARCHAR(20)	CHECK IN ('Active','Inactive','Suspended')

Products Table — 10 columns

The Products table represents the bag catalog. The IsCustomizable flag indicates whether a product can be personalized. UnitPrice is protected by a check constraint to prevent zero or negative values from entering the system.

Column	Type	Constraint
ProductID	VARCHAR(10)	PRIMARY KEY
ProductName	VARCHAR(100)	NOT NULL

Column	Type	Constraint
Description	VARCHAR(255)	NOT NULL
UnitPrice	DECIMAL(10,2)	NOT NULL, CHECK > 0
Size	VARCHAR(20)	CHECK IN ('Small','Medium','Large','Extra Large')
Color	VARCHAR(50)	NOT NULL
Material	VARCHAR(50)	NOT NULL
CreationDate	DATE	DEFAULT GETDATE()
IsCustomizable	BIT	DEFAULT 1

Orders Table — 8 columns

The Orders table tracks each purchase made by a customer. OrderStatus is controlled by a CHECK constraint that enforces the official lifecycle: Pending → Processing → Shipped → Delivered (or Cancelled). Shipping and delivery dates are nullable because they are filled in as the order progresses.

Column	Type	Constraint
OrderID	VARCHAR(12)	PRIMARY KEY
CustomerID	VARCHAR(10)	FK → Customers, NOT NULL
OrderDate	DATE	NOT NULL, DEFAULT GETDATE()
TotalAmount	DECIMAL(10,2)	NOT NULL, CHECK > 0
OrderStatus	VARCHAR(20)	CHECK IN ('Pending','Processing','Shipped','Delivered','Cancelled')
ShippingAddress	VARCHAR(255)	NOT NULL
ShippingDate	DATE	NULL
DeliveryDate	DATE	NULL

OrderItems Table — 6 columns

OrderItems acts as the junction table between Orders and Products. A single order can contain multiple items, each referencing a different product. The Subtotal column is a computed (persisted) column — SQL Server calculates it automatically as Quantity × UnitPrice, so it never needs to be inserted or updated manually.

Column	Type	Constraint
OrderItemID	VARCHAR(15)	PRIMARY KEY
OrderID	VARCHAR(12)	FK → Orders, NOT NULL
ProductID	VARCHAR(10)	FK → Products, NOT NULL
Quantity	INT	NOT NULL, CHECK > 0
UnitPrice	DECIMAL(10,2)	NOT NULL, CHECK > 0
Subtotal	DECIMAL (computed)	AS (Quantity * UnitPrice) PERSISTED

Payments Table — 7 columns

The Payments table records the financial transaction for each order. Because each order can have at most one payment, the OrderID foreign key also carries a UNIQUE constraint. The TransactionReference must also be unique across the table, preventing duplicate payment records for the same transaction.

Column	Type	Constraint
PaymentID	VARCHAR(12)	PRIMARY KEY
OrderID	VARCHAR(12)	FK → Orders, UNIQUE, NOT NULL
PaymentDate	DATE	NOT NULL, DEFAULT GETDATE()
PaymentAmount	DECIMAL(10,2)	NOT NULL, CHECK > 0
PaymentMethod	VARCHAR(20)	CHECK IN ('BankTransfer','Cash','MobilePayment','Cheque')
TransactionReference	VARCHAR(50)	UNIQUE, NOT NULL
PaymentStatus	VARCHAR(20)	CHECK IN ('Pending','Completed','Failed')

3. DDL — CREATE TABLE Statements

The following SQL statements create all five tables in SQL Server. Each statement includes the full set of constraints discussed in the previous section. These scripts can be run directly in SSMS to build the database from scratch.

Customers Table

```
CREATE TABLE Customers (  
    CustomerID        VARCHAR 10    PRIMARY KEY  
    FirstName         VARCHAR 50    NOT NULL  
    LastName          VARCHAR 50    NOT NULL  
    Email             VARCHAR 100    UNIQUE NOT NULL  
    PhoneNumber       VARCHAR 15    NOT NULL  
    Address           VARCHAR 255    NOT NULL  
    City              VARCHAR 50    NOT NULL  
    RegistrationDate  DATE          DEFAULT GETDATE  
    CustomerType      VARCHAR 20    CHECK (CustomerType IN ('Retail' 'Wholesale'  
    'Individual'))  
    Status            VARCHAR 20    CHECK (Status IN ('Active' 'Inactive'  
    'Suspended'))  
)
```

Products Table

```
CREATE TABLE Products (  
    ProductID        VARCHAR 10    PRIMARY KEY  
    ProductName       VARCHAR 100    NOT NULL  
    Description       VARCHAR 255    NOT NULL  
    UnitPrice        DECIMAL 10 2    NOT NULL CHECK (UnitPrice > 0  
    Size             VARCHAR 20    CHECK (Size IN ('Small' 'Medium' 'Large'  
    'Extra Large'))  
    Color            VARCHAR 50    NOT NULL  
    Material          VARCHAR 50    NOT NULL  
    CreationDate     DATE          DEFAULT GETDATE  
    IsCustomizable   BIT          DEFAULT 1  
)
```

Orders Table

```
CREATE TABLE Orders (
  OrderID          VARCHAR 12      PRIMARY KEY
  CustomerID       VARCHAR 10      NOT NULL
                                FOREIGN KEY REFERENCES Customers CustomerID
  OrderDate        DATE            NOT NULL DEFAULT GETDATE()
  TotalAmount      DECIMAL 10 2    NOT NULL CHECK (TotalAmount > 0)
  OrderStatus      VARCHAR 20      CHECK (OrderStatus IN (
                                'Pending' 'Processing' 'Shipped'
                                'Delivered' 'Cancelled'))
  ShippingAddress  VARCHAR 255     NOT NULL
  ShippingDate     DATE            NULL
  DeliveryDate     DATE            NULL
)
```

OrderItems Table

```
CREATE TABLE OrderItems (
  OrderItemID      VARCHAR 15      PRIMARY KEY
  OrderID          VARCHAR 12      NOT NULL
                                FOREIGN KEY REFERENCES Orders OrderID
  ProductID        VARCHAR 10      NOT NULL
                                FOREIGN KEY REFERENCES Products ProductID
  Quantity         INT             NOT NULL CHECK (Quantity > 0)
  UnitPrice        DECIMAL 10 2    NOT NULL CHECK (UnitPrice > 0)
  Subtotal         AS (Quantity * UnitPrice) PERSISTED
)
```

Payments Table

```
CREATE TABLE Payments (
  PaymentID        VARCHAR 12      PRIMARY KEY
  OrderID          VARCHAR 12      NOT NULL UNIQUE
                                FOREIGN KEY REFERENCES Orders OrderID
  PaymentDate      DATE            NOT NULL DEFAULT GETDATE()
  PaymentAmount    DECIMAL 10 2    NOT NULL CHECK (PaymentAmount > 0)
  PaymentMethod     VARCHAR 20      CHECK (PaymentMethod IN (
                                'BankTransfer' 'Cash'
                                'MobilePayment' 'Cheque'))
  TransactionReference VARCHAR 50    NOT NULL UNIQUE
  PaymentStatus     VARCHAR 20      CHECK (PaymentStatus IN (
                                'Pending' 'Completed' 'Failed'))
)
```

4. Sample Data

Representative INSERT statements that populate the database with test data. All rows satisfy the constraints defined above and respect referential integrity — that is, Orders reference existing Customers, OrderItems reference existing Orders and Products, and Payments reference existing Orders.

Customers — 5 rows

Five customers from different Ethiopian cities and covering all three customer types.

```
INSERT INTO Customers
  CustomerID  FirstName  LastName  Email  PhoneNumber
  Address  City  CustomerType  Status
VALUES
  'CUST001'  'Abebe'  'Desta'  'abebe@email.com'  '0911001001'
  'Bole, Addis Ababa'  'Addis Ababa'  'Retail'  'Active'
  'CUST002'  'Almaz'  'Mekonnen'  'almaz@email.com'  '0922002002'
  'Piassa, Addis Ababa'  'Addis Ababa'  'Wholesale'  'Active'
  'CUST003'  'Getnet'  'Abate'  'getnet@email.com'  '0933003003'
  'Kezira, Dire Dawa'  'Dire Dawa'  'Individual'  'Active'
  'CUST004'  'Marta'  'Girma'  'marta@email.com'  '0944004004'
  'Megenagna, Addis Ababa'  'Addis Ababa'  'Retail'  'Active'
  'CUST005'  'Kebede'  'Solomon'  'kebede@email.com'  '0955005005'
  'Tabor, Hawassa'  'Hawassa'  'Wholesale'  'Active'
```

Products — 5 rows

Five paper bag products spanning all available sizes and materials.

```

INSERT INTO Products
  ProductID ProductName Description UnitPrice
  Size Color Material IsCustomizable
VALUES
  'PROD001' 'Standard Grocery Bag' 'General purpose grocery bag'
25.00 'Medium' 'Brown' 'Kraft Paper' 1
  'PROD002' 'Premium Boutique Bag' 'High-end retail shopping
bag' 35.50 'Small' 'White' 'Coated Paper' 1
  'PROD003' 'Restaurant Bag Large' 'Takeaway food delivery
bag' 40.00 'Large' 'Kraft' 'Virgin Kraft' 1
  'PROD004' 'Event Gift Bag' 'Decorative bag for events and
gifts' 30.00 'Small' 'Cream' 'Recycled Paper' 1
  'PROD005' 'Extra Large Wholesale Bag' 'Bulk wholesale large
bag' 50.00 'Extra Large' 'Brown' 'Kraft Paper' 0

```

Orders — 4 rows

Four orders at different stages of the lifecycle, linked to existing customers.

```

INSERT INTO Orders
  OrderID CustomerID OrderDate TotalAmount
  OrderStatus ShippingAddress DeliveryDate
VALUES
  'ORD202505001' 'CUST001' '2025-05-20' 2500.00 'Delivered' 'Bole, Addis
Ababa' '2025-05-25'
  'ORD202505002' 'CUST002' '2025-05-21' 7500.00 'Shipped' 'Piassa, Addis
Ababa' NULL
  'ORD202505003' 'CUST003' '2025-05-22' 1420.00 'Processing' 'Kezira, Dire
Dawa' NULL
  'ORD202505004' 'CUST004' '2025-05-23' 3500.00 'Pending' 'Megenagna,
Addis Ababa' NULL

```

OrderItems — 3 rows

Three line items linking orders to specific products with quantities and unit prices. The Subtotal column is automatically computed — it does not need to be supplied.

```

INSERT INTO OrderItems
  OrderItemID OrderID ProductID Quantity UnitPrice
VALUES
  'ORDITEM-001-1' 'ORD202505001' 'PROD001' 100 25.00
  'ORDITEM-002-1' 'ORD202505002' 'PROD005' 150 50.00
  'ORDITEM-003-1' 'ORD202505003' 'PROD002' 40 35.50

```

Payments — 3 rows

Three payment records using different payment methods — one bank transfer, one mobile payment, and one cash transaction.

```
INSERT INTO Payments
  PaymentID OrderID PaymentDate PaymentAmount
  PaymentMethod TransactionReference PaymentStatus
VALUES
  'PAY202505001' 'ORD202505001' '2025-05-20' 2500.00
  'BankTransfer' 'TXN-CBE-20250520-001' 'Completed'
  'PAY202505002' 'ORD202505002' '2025-05-21' 7500.00
  'MobilePayment' 'TXN-TELEBIRR-20250521-002' 'Completed'
  'PAY202505003' 'ORD202505003' '2025-05-22' 1420.00
  'Cash' 'TXN-CASH-20250522-003' 'Pending'
```

5. Database Views

Views are saved SELECT queries that present data in a useful, pre-joined format without storing any additional data. The five views below serve as ready-made reports for business analysis — covering customer activity, product performance, order tracking, and payment analytics.

vw_CustomerOrderSummary

This view produces a per-customer summary showing how many orders each customer has placed, how much they have spent in total, and when their most recent order was. It uses a LEFT JOIN so that customers who have not yet placed any orders are still included in the results.

```
CREATE VIEW vw_CustomerOrderSummary AS
SELECT
  c CustomerID
  CONCAT c FirstName ' ' c LastName AS CustomerName
  c Email
  c CustomerType
  COUNT o OrderID AS TotalOrders
  SUM o TotalAmount AS TotalSpent
  MAX o OrderDate AS LastOrderDate
FROM Customers c
LEFT JOIN Orders o ON c CustomerID = o CustomerID
GROUP BY c CustomerID c FirstName c LastName c Email c CustomerType
```

vw_ProductSalesReport

This view aggregates sales data at the product level, showing total units sold, total revenue generated, the number of distinct orders that included the product, and the average quantity ordered per transaction.

```

CREATE VIEW vw_ProductSalesReport AS
SELECT
    p ProductID
    p ProductName
    p UnitPrice
    SUM oi Quantity AS TotalUnitsSold
    SUM oi Subtotal AS TotalRevenue
    COUNT DISTINCT oi OrderID AS OrderCount
    ROUND AVG oi Quantity 2 AS AvgQuantityPerOrder
FROM Products p
LEFT JOIN OrderItems oi ON p ProductID = oi ProductID
GROUP BY p ProductID p ProductName p UnitPrice

```

vw_OrderStatusTracking

This view gives a full lifecycle snapshot for every order — including the customer name, the number of distinct products in the order, the total quantity of items, and how many days the order has been in the system. It is useful for identifying orders that may be stuck or overdue.

```

CREATE VIEW vw_OrderStatusTracking AS
SELECT
    o OrderID
    CONCAT c FirstName ' ' c LastName AS CustomerName
    o OrderDate
    o OrderStatus
    o TotalAmount
    COUNT oi ProductID AS ItemCount
    SUM oi Quantity AS TotalQuantity
    DATEDIFF DAY o OrderDate GETDATE() AS DaysInSystem
FROM Orders o
JOIN Customers c ON o CustomerID = c CustomerID
LEFT JOIN OrderItems oi ON o OrderID = oi OrderID
GROUP BY o OrderID c FirstName c LastName
         o OrderDate o OrderStatus o TotalAmount

```

vw_PaymentStatusReport

This view groups payments by their status (Pending, Completed, or Failed) and reports the count, total value, average value, and number of distinct payment methods used within each group. It gives a quick financial health overview.

```

CREATE VIEW vw_PaymentStatusReport AS
SELECT
    p PaymentStatus
    COUNT p PaymentID AS TotalPayments
    SUM p PaymentAmount AS TotalAmount
    ROUND AVG p PaymentAmount 2 AS AvgPaymentAmount
    COUNT DISTINCT p PaymentMethod AS PaymentMethodsUsed
FROM Payments p
GROUP BY p PaymentStatus

```

vw_FullOrderDetails

This is the most comprehensive view, joining all five tables into a single result. Each row shows a complete picture of one order line item — the customer who placed it, what was ordered, the quantity and price, and the payment status. It is the primary view used for detailed reporting and auditing.

```

CREATE VIEW vw_FullOrderDetails AS
SELECT
    o OrderID
    CONCAT c FirstName ' ' c LastName AS CustomerName
    c CustomerType
    o OrderDate
    o OrderStatus
    p ProductName
    oi Quantity
    oi UnitPrice
    oi Subtotal
    pay PaymentMethod
    pay PaymentStatus
FROM Orders o
JOIN Customers c ON o CustomerID = c CustomerID
LEFT JOIN OrderItems oi ON o OrderID = oi OrderID
LEFT JOIN Products p ON oi ProductID = p ProductID
LEFT JOIN Payments pay ON o OrderID = pay OrderID

```

6. Stored Procedures

Stored procedures encapsulate business logic inside the database, preventing direct ad-hoc modifications to the tables. The seven procedures below cover all CRUD operations — inserting, updating, and reading data — while also enforcing rules such as validating that a customer exists before an order can be created.

sp_InsertCustomer — Register a new customer

Accepts all required customer details as parameters and inserts a new row into the Customers table. The Status is always set to 'Active' on registration — it cannot be set to anything else through this procedure.

```
CREATE PROCEDURE sp_InsertCustomer
    @CustomerID VARCHAR 10
    @FirstName  VARCHAR 50
    @LastName   VARCHAR 50
    @Email      VARCHAR 100
    @PhoneNumber VARCHAR 15
    @Address    VARCHAR 255
    @City       VARCHAR 50
    @CustomerType VARCHAR 20
AS
BEGIN
    INSERT INTO Customers
        CustomerID FirstName LastName Email PhoneNumber
        Address City CustomerType Status
    VALUES
        @CustomerID @FirstName @LastName @Email @PhoneNumber
        @Address @City @CustomerType 'Active'
    PRINT 'Customer registered successfully.'
END
```

sp_UpdateCustomer — Update customer contact or status

All parameters except CustomerID are optional (they default to NULL). The ISNULL function ensures that only the fields actually provided by the caller are changed — any NULL parameter simply leaves the existing value in place. This makes partial updates safe and straightforward.

```

CREATE PROCEDURE sp_UpdateCustomer
    @CustomerID VARCHAR 10
    @Email       VARCHAR 100 = NULL
    @PhoneNumber VARCHAR 15  = NULL
    @Address     VARCHAR 255 = NULL
    @Status     VARCHAR 20  = NULL
AS
BEGIN
    UPDATE Customers
    SET
        Email       = ISNULL @Email       Email
        PhoneNumber = ISNULL @PhoneNumber PhoneNumber
        Address     = ISNULL @Address     Address
        Status      = ISNULL @Status      Status
    WHERE CustomerID = @CustomerID
    PRINT 'Customer updated.'
END

```

sp_CreateOrder — Create an order with customer validation

Before inserting the order, this procedure verifies that the specified CustomerID actually exists in the Customers table. If not, it prints an error message and exits early with RETURN. This prevents orphaned orders from being created even if the foreign key constraint were somehow bypassed.

```

CREATE PROCEDURE sp_CreateOrder
    @OrderID      VARCHAR 12
    @CustomerID   VARCHAR 10
    @TotalAmount  DECIMAL 10 2
    @ShippingAddress VARCHAR 255
AS
BEGIN
    IF NOT EXISTS (SELECT 1 FROM Customers WHERE CustomerID = @CustomerID)
    BEGIN
        PRINT 'Error: Customer does not exist.'
        RETURN
    END

    INSERT INTO Orders
        OrderID CustomerID TotalAmount OrderStatus ShippingAddress
    VALUES
        (@OrderID @CustomerID @TotalAmount 'Pending' @ShippingAddress)
    PRINT 'Order created successfully.'
END

```

sp_AddOrderItem — Add an item and auto-update the order total

Inserts a new line item into OrderItems and then immediately updates the parent order's TotalAmount by adding the new item's value. This keeps the Orders table in sync with the actual items without requiring a separate update from the caller.

```
CREATE PROCEDURE sp_AddOrderItem
    @OrderItemID VARCHAR 15
    @OrderID      VARCHAR 12
    @ProductID    VARCHAR 10
    @Quantity     INT
    @UnitPrice    DECIMAL 10 2
AS
BEGIN
    INSERT INTO OrderItems
        OrderItemID OrderID ProductID Quantity UnitPrice
    VALUES
        (@OrderItemID @OrderID @ProductID @Quantity @UnitPrice)

    -- Keep Orders.TotalAmount in sync with actual line items
    UPDATE Orders
    SET TotalAmount = TotalAmount + @Quantity * @UnitPrice
    WHERE OrderID = @OrderID
    PRINT 'Item added to order.'
END
```

sp_RecordPayment — Record payment and advance order status

Inserts a payment record with a status of 'Completed', and simultaneously updates the linked order's status to 'Processing'. This ensures that a confirmed payment automatically moves the order forward in the lifecycle without any additional steps.

```

CREATE PROCEDURE sp_RecordPayment
    @PaymentID          VARCHAR 12
    @OrderID            VARCHAR 12
    @PaymentAmount      DECIMAL 10 2
    @PaymentMethod      VARCHAR 20
    @TransactionReference VARCHAR 50
AS
BEGIN
    INSERT INTO Payments
        PaymentID OrderID PaymentAmount
        PaymentMethod TransactionReference PaymentStatus
    VALUES
        (@PaymentID @OrderID @PaymentAmount
        @PaymentMethod @TransactionReference 'Completed')

    UPDATE Orders
    SET OrderStatus = 'Processing'
    WHERE OrderID = @OrderID
    PRINT 'Payment recorded successfully.'
END

```

sp_UpdateOrderStatus — Advance an order through the lifecycle

Allows the order status to be changed to any valid value, and optionally updates the shipping or delivery date at the same time. Like sp_UpdateCustomer, it uses ISNULL so that date fields not supplied by the caller remain unchanged.

```

CREATE PROCEDURE sp_UpdateOrderStatus
    @OrderID          VARCHAR 12
    @OrderStatus      VARCHAR 20
    @ShippingDate     DATE = NULL
    @DeliveryDate     DATE = NULL
AS
BEGIN
    UPDATE Orders
    SET
        OrderStatus = @OrderStatus
        ShippingDate = ISNULL @ShippingDate ShippingDate
        DeliveryDate = ISNULL @DeliveryDate DeliveryDate
    WHERE OrderID = @OrderID
    PRINT 'Order status updated.'
END

```

sp_GetCustomerOrderDetails — Parameterized report by CustomerID

Returns a summary of all orders placed by a specific customer, including the item count and total quantity per order. This is a read-only procedure used for customer-facing reporting and account management.

```
CREATE PROCEDURE sp_GetCustomerOrderDetails
    @CustomerID VARCHAR 10
AS
BEGIN
    SELECT
        o OrderID
        o OrderDate
        o OrderStatus
        o TotalAmount
        COUNT oi ProductID AS ItemCount
        SUM oi Quantity     AS TotalQuantity
    FROM Orders o
    LEFT JOIN OrderItems oi ON o OrderID = oi OrderID
    WHERE o CustomerID = @CustomerID
    GROUP BY o OrderID o OrderDate o OrderStatus o TotalAmount
END
```

7. Built-in SQL Functions

This section demonstrates how SQL Server's built-in functions are used throughout the EcoBag Addis system. The examples are organized into four categories: string functions, aggregate functions, date functions, and conditional functions.

String Functions — CONCAT, UPPER, LOWER, LEN

String functions are used primarily for formatting output. For example, customer names are stored as separate first and last name columns, so CONCAT is used whenever a full name needs to be displayed. UPPER and LOWER normalize text for display or comparison purposes.

```
SELECT
    CustomerID
    UPPER CONCAT FirstName ' ' LastName AS FullNameUpper
    LOWER Email AS EmailLower
    LEN PhoneNumber AS PhoneLength
FROM Customers
```

Aggregate Functions — COUNT, SUM, AVG, MAX, MIN, ROUND

Aggregate functions are the backbone of the reporting views. They collapse many rows into a single summary value per group. This example calculates sales statistics for each product across all order items.

```
SELECT
    ProductID
    COUNT ProductID AS TotalOrders
    SUM Quantity AS UnitsSold
    ROUND AVG Subtotal 2 AS AvgOrderValue
    MAX Subtotal AS HighestOrder
    MIN Subtotal AS LowestOrder
FROM OrderItems
GROUP BY ProductID
```

Date Functions — GETDATE, DATEDIFF, YEAR, MONTH

Date functions are used to calculate how long ago an order was placed and to filter orders by the current year. DATEDIFF is particularly important for the order tracking view, where it calculates how many days an order has been in the system.

```
SELECT
    OrderID
    OrderDate
    DATEDIFF DAY OrderDate GETDATE() AS DaysAgo
    YEAR OrderDate AS OrderYear
    MONTH OrderDate AS OrderMonth
FROM Orders
WHERE YEAR OrderDate = YEAR GETDATE()
```

Conditional Functions — CASE, ISNULL

CASE expressions convert coded status values into human-readable descriptions for reporting. ISNULL handles nullable date columns gracefully — instead of displaying a blank cell, it returns a meaningful text value such as 'Not Yet Delivered'.

```
SELECT
    OrderID
    OrderStatus
    CASE
        WHEN OrderStatus = 'Delivered' THEN 'Completed'
        WHEN OrderStatus = 'Shipped' THEN 'In Transit'
        WHEN OrderStatus = 'Processing' THEN 'Being Prepared'
        WHEN OrderStatus = 'Pending' THEN 'Awaiting Processing'
        ELSE 'Other'
    END AS StatusDescription
    ISNULL CAST DeliveryDate AS VARCHAR) 'Not Yet Delivered' AS DeliveryInfo
FROM Orders
```